

Metro Ticket Booking System

Solution Architecture Specification (Requirement & Design Phase)

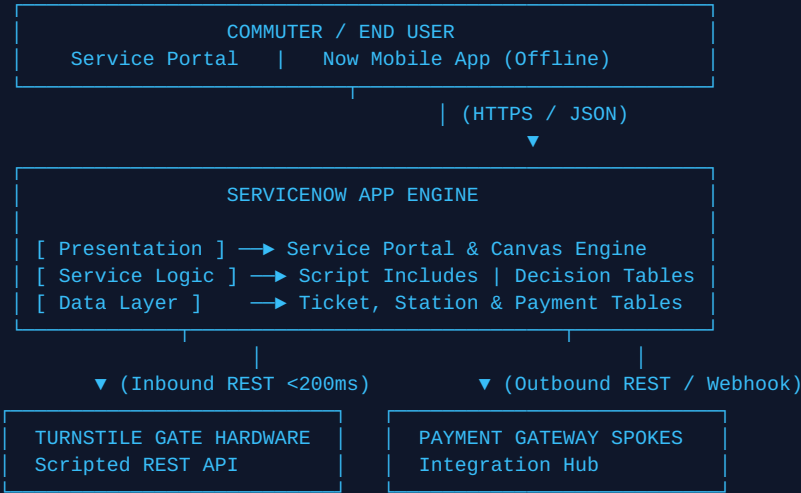
PLATFORM TARGET
ServiceNow App Engine

ARCHITECTURE SCOPE
Scoped App (x_snc_metro_book)

DOCUMENT VERSION
v1.0 Final

1. High-Level Architectural Framework

The solution utilizes a 3-tier ServiceNow Scoped Application Architecture bridging physical station turnstiles, mobile/web interfaces, and platform background processes.



2. Layer-by-Layer Component Specification

A. Presentation & Access Layer

- **Service Portal:** Dedicated portal for commuters to select routes, check dynamic fare tables, and execute transactions.
- **Now Mobile App Engine:** Mobile interface leveraging local client storage to store active ticket tokens, allowing offline QR rendering underground without cellular coverage.
- **Next Experience Workspace:** Operational workspace for station managers to track volume, manage stations, and handle transaction exceptions.

B. Business Logic & Processing Layer

- **Script Includes:** Server-side JavaScript engine (MetroFareCalculator, MetroSecurityEngine) providing cryptographically signed HMAC/TOTP QR tokens and validation logic.
- **Decision Tables Engine:** Manages tariff rules (peak multipliers, distance logic, category discounts) without hardcoding business rules.
- **Flow Designer Workflows:** Orchestrates post-payment execution, ticket generation, SMS/WhatsApp messaging, and failure handling.

C. Inbound Turnstile Integration (<200ms Validation)

- **Scripted REST API (/api/x_metro/gate_validation/scan):** High-performance endpoint querying indexed composite keys on sn_metro_ticket to return sub-150ms verification decisions.

- **Async State Offloading:** State transitions (e.g., Active to In-Transit) execute asynchronously via Business Rules to ensure zero delay at hardware gates.

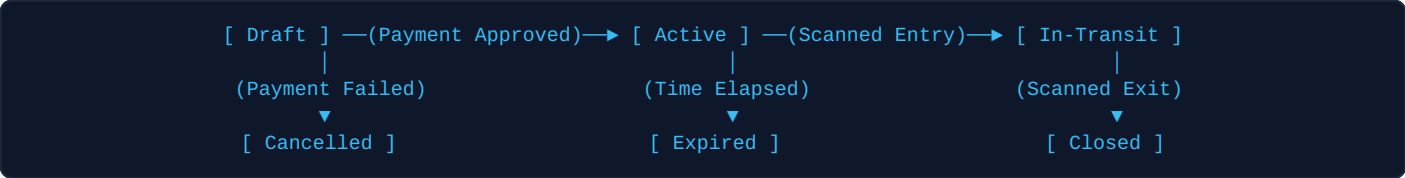
D. Outbound Integrations & Financial Operations

- **Integration Hub Spokes:** Connects securely with external payment processors (Stripe, Razorpay, UPI) to handle transactions.
- **Idempotent Webhooks & Auto-Refund Flow:** Validates payment callbacks via transaction keys and triggers an automated refund workflow if ticket activation fails within 60 seconds.

3. Data Architecture & Entity Relationships

Table Name	Extended Table	Primary Index	Key Fields & Purpose
sn_metro_station	Base Table	sys_id, code	name, code, line_color, active Station master database.
sn_metro_fare_matrix	Base Table	Composite (origin, dest)	origin_station, dest_station, base_fare Distance & tariff pricing matrix.
sn_metro_ticket	task	Composite (sys_id, state, qr_token)	number, commuter, origin, destination, state, qr_token Core ticket state lifecycle.
sn_metro_payment	Base Table	transaction_id	ticket, transaction_id, amount, payment_status Financial transaction audit log.

4. Ticket Lifecycle State Machine



1. **Draft:** Ticket created upon selection; awaiting payment authorization.
2. **Active:** Payment confirmed; TOTP dynamic QR token rendered on commuter device.
3. **In-Transit:** Commuter scanned at entry gate; timestamp and entry station recorded.
4. **Closed:** Commuter scanned at destination exit gate; ticket lifecycle completed.
5. **Expired:** Unused ticket exceeds validity duration (e.g., 3 hours); automatically marked by background engine.

5. Key Non-Functional Requirements (NFRs)

- **Latency Goal:** Turnstile verification executed under 200ms end-to-end.
- **Security Engine:** TOTP dynamic QR payload rotating every 30 seconds to prevent screenshot sharing.
- **Application Scope:** Purely scoped application (x_snc_metro_book) ensuring platform security and zero conflict with ITIL process applications.